AZIZA

Russian Cyrillic Test — Readiness Plan

Field	Value
Engineer	Chris — Novatech
Test target	Russian Cyrillic text comprehension — NO voice/audio
Deadline	Tomorrow evening
Model	PersonaPlex-7B + Russian QLoRA adapter (r=16)
Endpoints	Cloudflare tunnel (client) + Direct Vast.ai port (internal)
Test URL	wss://shed-dayton-educational-copyrighted.trycloudflare.com
Direct port	Vast.ai H100 — port 8020 (serve_russian_test.py)
Scope	Text-in / text-out ONLY — VibeVoice pipeline NOT required

- OBJECTIVE: Ivan sends Russian Cyrillic text → Aziza responds correctly in Russian.
- SKIP: VibeVoice audio generation, ASR, TTS, Asterisk — not needed for this test.
- TONIGHT: Train Russian QLoRA adapter (12–18 hrs on H100).
- TOMORROW MORNING: Eval + deploy serve_russian_test.py on port 8020.
- TOMORROW AFTERNOON: Smoke test via /test/cyrillic endpoint before Ivan connects.

TIMELINE — TONIGHT → TOMORROW EVENING

When	Task	Success Criterion	Status
Tonight ~now	Fix moshi-worker blocker (huggingface-hub)	Confirm pm2 status shows 0 restarts	TONIGHT
Tonight ~now	Verify GPU is clear — VRAM < 5GB used	nvidia-smi before starting training	TONIGHT
Tonight ~now	Start ru_colloquial training run	~12–18 hrs on H100	TONIGHT
Tomorrow morning	Check training loss curve in TensorBoard	p6006 — confirm loss decreasing	TOMORROW
Tomorrow morning	Training completes — adapter saved	adapters/ru_colloquial/ exists	TOMORROW
Tomorrow morning	Run eval_russian.py — all gates pass	≥80% pass rate + Cyrillic in responses	TOMORROW
Tomorrow midday	Deploy serve_russian_test.py port 8020	uvicorn + systemd service	TOMORROW
Tomorrow midday	Hit GET /test/cyrillic — all 5 tests pass	ready_for_demo: true in response	TOMORROW
Tomorrow midday	Verify Cloudflare tunnel routes to port 8020	curl tunnel URL /health	TOMORROW
Tomorrow evening	Client test — Ivan sends Russian text	Response in Cyrillic, coherent, < 2s	TOMORROW

STEP 0 — FIX BLOCKER FIRST (< 5 minutes)

The aziza-moshi-worker crash-loop MUST be fixed before training starts or the training process will hit the same huggingface-hub import error.

```
source /root/personaplex-env/bin/activate
pip install "huggingface-hub>=1.5.0" "transformers>=4.40.0" --force-reinstall
pip check
python3 -c "from huggingface_hub import login; print('OK')"
pm2 restart aziza-moshi-worker && pm2 status
```

- Expected: aziza-moshi-worker shows "online" with restarts count frozen.
- If torch/vllm conflict: pip install "huggingface-hub>=1.5.0" --no-deps first.

STEP 1 — TRAIN RUSSIAN QLORA ADAPTER (tonight, ~12–18 hrs)

Environment check before starting

```
nvidia-smi # confirm VRAM free before training
```

```
df -h /root # need ≥ 80GB free for checkpoints
head -1 data/aziza-bilingual/aziza-bilingual.jsonl | python3 -m json.tool
```

Check the JSONL output. The training script expects one of these field patterns:

Field pattern	Example
messages (list)	[{"role":"user","content":"[redacted]"}, {"role":"assistant","content":"..."}]
instruction + response	{"instruction": "[redacted]", "response": "[redacted]!"}
text (raw)	{"text": "<|user|>\n[redacted]\n<|assistant|>\n[redacted]!"}
language field	"language": "ru" — used for filtering (optional but recommended)
register field	"register": "colloquial" — used for filtering (optional)

Launch training — run in screen or tmux (it runs overnight)

```
# Start a persistent session so SSH disconnect does not kill it
screen -S aziza-train
# OR
tmux new -s aziza-train
# Then inside the session:
cd /workspace/aziza-web/fine-tuning
source /root/personaplex-env/bin/activate
export HF_TOKEN=hf_your_token_here
# Copy training scripts to the fine-tuning directory
cp train_russian.py eval_russian.py serve_russian_test.py .
# Start TensorBoard (monitor from browser on port 6006)
tensorboard --logdir runs --host 0.0.0.0 --port 6006 &
# Launch training — colloquial register first (fastest to verify)
python3 train_russian.py --register colloquial \
--dataset_path data/aziza-bilingual/aziza-bilingual.jsonl \
--output_base /root/aziza/adapters
# Detach from screen/tmux: Ctrl+A D (screen) or Ctrl+B D (tmux)
```

Monitor training progress

```
# Reattach: screen -r aziza-train OR tmux attach -t aziza-train
watch -n 10 nvidia-smi # GPU should be 90-100%
tail -f runs/ru_colloquial/trainer_log.jsonl # live loss values
```

What to watch	Expected value
GPU utilisation	90–100% — if < 70%, dataloader is bottlenecking
VRAM usage	35–45 GB — if > 75GB, reduce batch_size to 2
Train loss epoch 1	~2.6–2.9

Train loss epoch 2	~1.8–2.1
Train loss epoch 3	~1.3–1.6 (early stop triggers if plateau)
Eval loss	Should track below train loss — if diverges, learning rate too high
OOM error	Add --batch_size 2 --grad_accum 8 and restart

STEP 2 — EVALUATE ADAPTER (tomorrow morning, ~15 min)

Run immediately after training completes. Do NOT deploy until eval passes.

```
python3 eval_russian.py \
--adapter /root/aziza/adapters/ru_colloquial/ \
--ttft_target_ms 180
```

Acceptance gates — ALL must pass before deployment

Gate	Criterion	Where to check	Status
Pass rate	≥ 8/10 test prompts return Cyrillic responses	eval_russian.py summary	VERIFY
RU Cyrillic %	≥ 80% of Russian prompts contain Cyrillic output	ru_prompts_cyrillic_pct	VERIFY
TTFT estimate	≤ 180ms per-token on H100 SXM	avg_ttft_est_ms	VERIFY
Adapter loads	PeftModel.from_pretrained() — no error	no RuntimeError/OOM	VERIFY
Persona prompt	Responds to "■■ — ■■■■■. ■■■■■■■■■■■■." in Russian	manual check response	VERIFY

■ Results saved to: /root/aziza/adapters/ru_colloquial/eval_results.json

■ If pass rate < 80%: check dataset format — most likely the filter is too strict.

```
Rerun with --register colloquial removed (trains on all RU data)
```

■ If TTFT > 180ms: this is an estimate. Real TTFT measured during serve test.

```
Proceed anyway — H100 real TTFT is typically lower than the batch estimate.
```

STEP 3 — DEPLOY TEXT TEST API (tomorrow midday)

3a. Set environment and start server

```
export HF_TOKEN=hf_your_token_here
export MODEL_ID=nvidia/personaplex-7b-v1
export ADAPTER_PATH=/root/aziza/adapters/ru_colloquial
export PORT=8020
```

```
uvicorn serve_russian_test:app --host 0.0.0.0 --port 8020 --workers 1
```

Use PM2 to keep it alive:

```
pm2 start "uvicorn serve_russian_test:app --host 0.0.0.0 --port 8020 --workers 1" \
--name aziza-russian-test --interpreter none
pm2 save
```

3b. Add port 8020 on Vast.ai

1. Go to Vast.ai console → your H100 instance → "Edit"
2. Under "Open Ports", add: 8020 (TCP)
3. Save — the direct URL will be: http://:
4. Note the mapped port number shown — it maps external → internal 8020

3c. Route Cloudflare tunnel to port 8020

The current tunnel points to port 8010 (NestJS). For the Russian test, either update the tunnel OR run a second tunnel specifically for the test API:

```
# Option A — new tunnel just for the Russian test endpoint
cloudflared tunnel --url http://localhost:8020
# Note the new URL it prints — send THIS to Ivan for the test
```

```
# Option B — update existing tunnel to proxy /test path to 8020
# Edit ~/.cloudflared/config.yml and add ingress rule for port 8020
```

3d. Smoke test before Ivan connects

```
# 1. Health check
curl https:///health
→ Expect: {"status":"ready","adapter":"ru_colloquial",...}

# 2. Auto Cyrillic readiness check — run this first
curl https:///test/cyrillic
→ Expect: {"all_pass":true,"ready_for_demo":true,...}

# 3. Manual Russian chat test
curl -X POST https:///chat \
-H "Content-Type: application/json" \
-d '{"message": "[redacted], [redacted]?"}'
→ Expect response in Cyrillic, has_cyrillic: true

# 4. Internal direct test (your machine)
curl -X POST http://:/chat \
-H "Content-Type: application/json" \
-d '{"message": "[redacted] [redacted] [redacted] [redacted]-[redacted]."}'
```

STEP 4 — CLIENT TEST PROTOCOL (tomorrow evening)

What Ivan needs to send the test. Keep it simple — POST /chat with JSON.

Send Ivan these details

Endpoint: POST https:///chat

Headers: Content-Type: application/json

Request body:

```
{
"message": "",
"temperature": 0.7
}
```

Response:

```
{
"response": "",
"has_cyrillic": true,
"output_tokens": 45,
"total_ms": 1240.5,
"adapter": "ru_colloquial"
}
```

Test prompt suggestions — give Ivan these

Prompt (Russian)	What it tests
■■■■■■! ■■ ■■■■■■■■■■ ■■■■■■■■ ■■■■■?	Basic Cyrillic comprehension + greeting
■■■ ■■■■■ ■■■■■■ ■ ■■■■ ■■ ■■■■■■■■ ■■■■■■■?	Persona awareness + capability description
■■■■■■■■ ■■■■■■■■ ■■■■■■ ■■■■■■■■ ■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■.	Complex Russian — academic vocabulary
■■■ ■■■■■■ ■■■■■■■ ■ ■■■■■■■■ ■ ■■■■■■■■■■.	Professional register — hospitality domain (IMIDUS relevant)
■■■■■■■■■ ■■■■ ■■■■-■■■■■■■ ■■ ■■■■■■■■■■■■.	Cross-language knowledge + cultural context

Pass criteria for client test

Criterion	Description	How to check	Status
Cyrillic output	Every response contains Cyrillic characters	has_cyrillic: true in JSON	VERIFY
Coherence	Response is grammatically correct Russian	Human review by Ivan	VERIFY
Response time	< 3 seconds wall-clock for a typical message	total_ms field in response	VERIFY
Persona	Identifies as ■■■■■ when asked	Check response to name prompt	VERIFY
No hallucination	Does not switch to English mid-response unexpectedly	Check 5 consecutive responses	VERIFY

WHAT WE ARE SKIPPING FOR THIS TEST

✗ VibeVoice audio generation — not needed, was causing the shard failure

✗ Asterisk ARI / telephony pipeline — Phase 5, after this test

✗ Speech-to-Text (Whisper) — text input only for this test

✗ TTS output — text response only, no audio synthesis

✗ Uzbek adapters — test Russian first, UZ follows same pipeline

✗ uz_professional / uz_academic / ru_academic adapters — train after ru_colloquial passes

■ Everything above is deferred — NOT cancelled. Once Ivan signs off on Russian text comprehension, we continue with remaining adapters + voice pipeline.

IF TRAINING FAILS OR RUNS OVER TIME

Fallback plan if the adapter is not ready by tomorrow evening:

Option A — Deploy base PersonaPlex-7B without adapter:

- Set ADAPTER_PATH="" in serve_russian_test.py
- Load base model only — PersonaPlex-7B has some multilingual ability
- Responses will be less Russian-native but still Cyrillic-capable
- Tell Ivan: "This is base model — fine-tuned adapter arriving Day+1"

Option B — Extend deadline by 4 hours:

- Training should complete by morning if started tonight
- Eval + deploy takes 30 min — push client test to late afternoon

Option C — Use a smaller model for faster training:

- Add --max_seq_len 256 --epochs 1 to get a faster adapter (~4-6 hrs)
- Quality will be lower but sufficient for a comprehension demo